Contribution Report

Team Member	Component(s)	Specific Tasks Completed	Testing/debugging	GitHub Contributions
Mason Bean	Linked list Implementation, Waiting List Management	Implemented the linked list structure for storing active reservations. Developed functions to create and insert reservation nodes, remove reservations, traverse the list, and display reservation information. Implemented the waiting-list queue for students waiting for unavailable resources, including enqueue/add, dequeue/remove, and display operations. Added handling for empty lists and queues.	Tested inserting multiple reservations and removing reservations from different positions. Verified that reservation records were displayed correctly during traversal. Tested adding several students to the waiting list and confirmed that students were removed in FIFO order. Debugged pointer updates, empty-structure cases, and incorrect links between nodes.	Worked on a dedicated feature branch for the linked-list and waiting-list functionality. Made commits for implementation, testing, and debugging with meaningful commit messages. Contributed the completed changes through the team's pull request.
Ryker Dunbar	Cancellation History (stack), Complexity Analysis, and the shared Reservation data structure	Wrote include/Reservation.h (the Reservation struct: id, studentId, studentName, resourceId, date), matching reservations.txt Wrote include/CancellationHistory.h: class declaration for a linked-node stack with push, pop, peek, isEmpty, size, and display. Implemented src/CancellationHistory.cpp: constructor, destructor (frees all nodes), push/pop/peek with empty-	Wrote tests/test_stack.cpp: covers empty-stack pop/peek returning false, three pushes, peek without removal, and correct LIFO pop order. Ran the test and confirmed all assertions passed. Compiled CancellationHistory.cpp cleanly with g++	Four commits to Rykers-Branch under rykerd123: "Added Reservation struct matching reservations.txt fields," "Added CancellationHistory stack header," "Implemented stack push, pop, peek, display," "Added unit test for cancellation history stack."

		stack handling, and display printing most-recent-first. Wrote the Complexity Analysis document (docs/complexity_analysis.docx), covering reservation insertion, removal, waiting-list processing, and undo cancellation		Pushed all commits to the remote repository.
Owen Crisp	Resource Management, Reservation Management	Implemented adding, removing, searching, and displaying resources; implemented creation and management of student reservations; connected reservations to resources; handled unavailable and invalid resources; updated reservation and resource information.,	Tested adding/removing resources, displaying resources, creating reservations, unavailable resources cases, and invalid inputs. Debugged resource and reservation updates.,	Used my dedicated branch and made commits for implementation, testing, and debugging. Merged it all after completion.

Short Paragraphs

Mason

I was responsible for developing the linked list for active reservations and the queue for managing waiting lists. I implemented the main operations for both structures, including Inserting, removing, traversing, and displaying reservation records, as well as adding, removing, and displaying students on the waiting list. I tested the components with multiple records and different edge cases, including empty structures, fixed-pointer issues, and ordering issues. I also contributed my code through GitHub using my separate branch and 4 commits under my GitHub username: MasonB0622.

Ryker

I was responsible for the cancellation history stack and the project's complexity analysis. I built the stack (push, pop, peek, isEmpty, size, display) as a linked list in CancellationHistory.h/.cpp, with a destructor that frees every node and safe handling when the stack is empty. I wrote a test that checks empty-stack behavior and confirms cancellations come back in the correct last-in-first-out order, and verified it passed. I also wrote Reservation.h, the shared data structure used across the project. For the complexity analysis, I read the actual linked list and queue code and explained the time complexity of reservation insertion, removal, waiting list processing, and undo cancellation with big o notation. All my work was committed and pushed to my branch under my GitHub account.

Owen

I was responsible for developing the resource management and reservation functionality. I implemented the features for adding, removing, searching for, and displaying available resources, as well as creating and managing student reservations. I worked with the shared Reservation structure to ensure reservation information was stored and handled correctly. I tested the functionality with multiple resources and reservations, including checking available resources, creating reservations, and handling invalid or unavailable resources. I also debugged issues with reservation data and resource updates to make sure the system behaved correctly. All my work was completed on my own GitHub branch, and I contributed to the changes in the final merge.

Additions

Mason

Implemented the hand-written merge sort for resources (sorting by name, no std::sort), and wrote all four reporting functions: Active Reservations, Resource Utilization, Most Requested Resources, and Waiting-List Statistics. Pushed to Mason's-Branch.

Owen

Replaced the single available/unavailable flag on Resource with per-date availability tracking, fixing the root architectural bug flagged in Milestone 1 feedback. Added date-conflict validation during reservation creation. Wired cancelReservation() to automatically promote the next matching student off the waiting list when a reservation is cancelled. Implemented search by reservation ID and by student name. Pushed to Owen's-branch.

Ryker

Restored the missing test_stack.cpp file flagged in Milestone 1 feedback. Performed the final system integration: pulled Mason's and Owen's work from their separate branches and merged it into a working, tested build. Wrote new automated tests covering the sort, reports, per-date availability, search, and waiting-list promotion logic. Verified the full build compiles on the UNT CSE CELL machines. Rewrote the README and wrote the dedicated user documentation file. Merged all work into main for final submission.